PEAT REWETTING PLANNING SYSTEM

System Architecture Document

Structure, deployment, runtime behaviour, security, performance and decisions of the system.

Project	Peat Rewetting Planning for 337 Peat Hydrological Units (KHG)
Document	System Architecture
Version	1.0, 1 October 2026
Author	Fairuz Akmal Pradana
Repository	github.com/fakmalpradana/pembasahan-gambut-337khg

Contents

Introduction

- Purpose
- Scope
- Audience
- Related documents

Architectural goals and constraints

- Goals
- Constraints

System context

Container view

Database component view

Data architecture and pipeline

- Sources
- Pipeline

Runtime scenarios

- Public read
- Authenticated edit through the API
- Review workflow

Deployment view

Cross-cutting concerns

- Security
- Performance (measured)
- Availability and recovery
- Observability

Architecture decisions

Risks and technical debt

Cost summary

Introduction

Purpose

This System Architecture Document (SAD) describes how the Peat Rewetting Planning system is built, deployed and operated. It explains the main structures, the reasons behind them and the measured quality of the result.

Scope

The system covers:

- A PostgreSQL 17 and PostGIS 3.5 database with seven data domains and full change history.
- Repeatable data pipelines that load four source datasets (about 3.8 million features).
- A standards-based web service (OGC API) built on pygeoapi, behind a Caddy reverse proxy.
- Deployment automation for Hetzner Cloud.

The web user interface shown in the design mockup is **not** part of this delivery. The database and API are designed so that the interface can be built on top of them.

Audience

Architects, developers, GIS engineers, operators and the team presenting the system.

Related documents

Document	Content
Domain-Driven Design	Business language, bounded contexts, aggregates, rules
Entity Relationship Diagram	Entities and relationships per context
Database Schema	Every table, column, constraint, index, trigger and function
Repository `docs/03_API_OGC.md`	API reference with examples (Indonesian)

Architectural goals and constraints

Goals

Goal	What it means in practice
Standards first	Any GIS tool must read the data without custom code: OGC API Features, Tiles and Processes.
Two editing channels	Analysts edit directly in QGIS (PostGIS connection) and through web or API clients, at the same time.
Full traceability	Every change is recorded with user, tool, time, old and new values, and plan version.
Low running cost	A demo costs about EUR 0.05 per hour; production should stay near EUR 1 per day.
Reproducible	The whole database can be rebuilt from source files with one command.
Open source only	PostgreSQL, PostGIS, pygeoapi, Caddy, GDAL, Python. No licence fees.

Constraints

- Source data arrives as shapefiles and an Esri File Geodatabase with inconsistent names, mixed units and duplicates.
- The largest layers hold close to one million polygons each, so queries must use spatial indexes.
- QGIS writes straight to the database, so auditing cannot live in application code.
- The team has no dedicated operations staff, so deployment must be scripted end to end.

System context

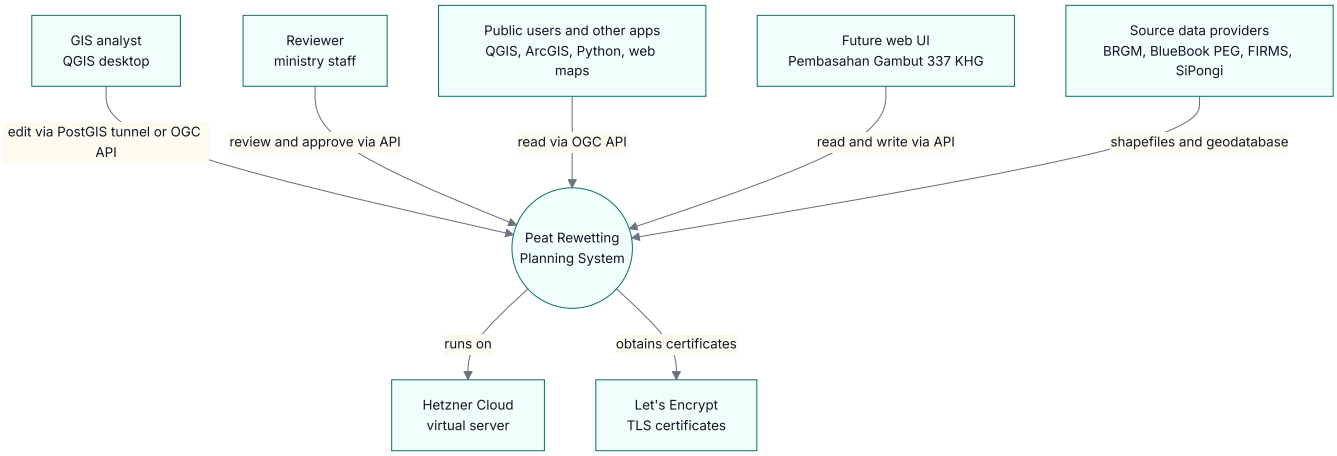


Figure 1. System context.

Container view

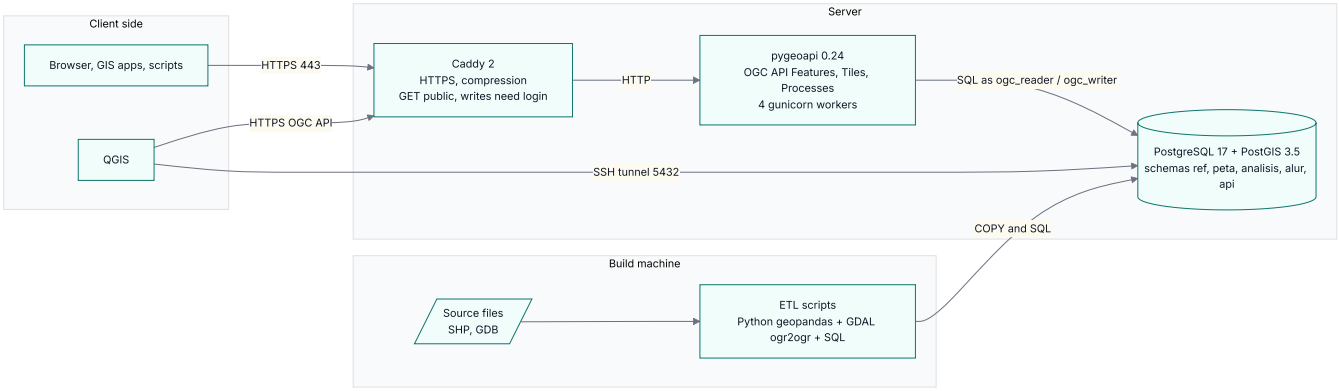


Figure 2. Containers and their connections.

Container	Technology	Responsibility
Reverse proxy	Caddy 2	TLS termination with automatic certificates, gzip and zstd, HTTP Basic authentication for every non-GET request except read-only processes
OGC API server	pygeoapi 0.24.0, Flask, gunicorn	19 feature collections, 8 vector tile sets, 6 processes, OpenAPI document, HTML views
Process plugins	Python module `gambut_processes.py`	Dashboard summary, validation, pump suggestion, submit and decide review, refresh statistics
Database	PostgreSQL 17.5, PostGIS 3.5.2	Storage, spatial indexes, triggers for audit, versioning and codes, validation and suggestion functions
ETL	Python 3, geopandas, pyogrio, GDAL 3, psql	Clean, stage, load, dissolve and finalise source data

Database component view

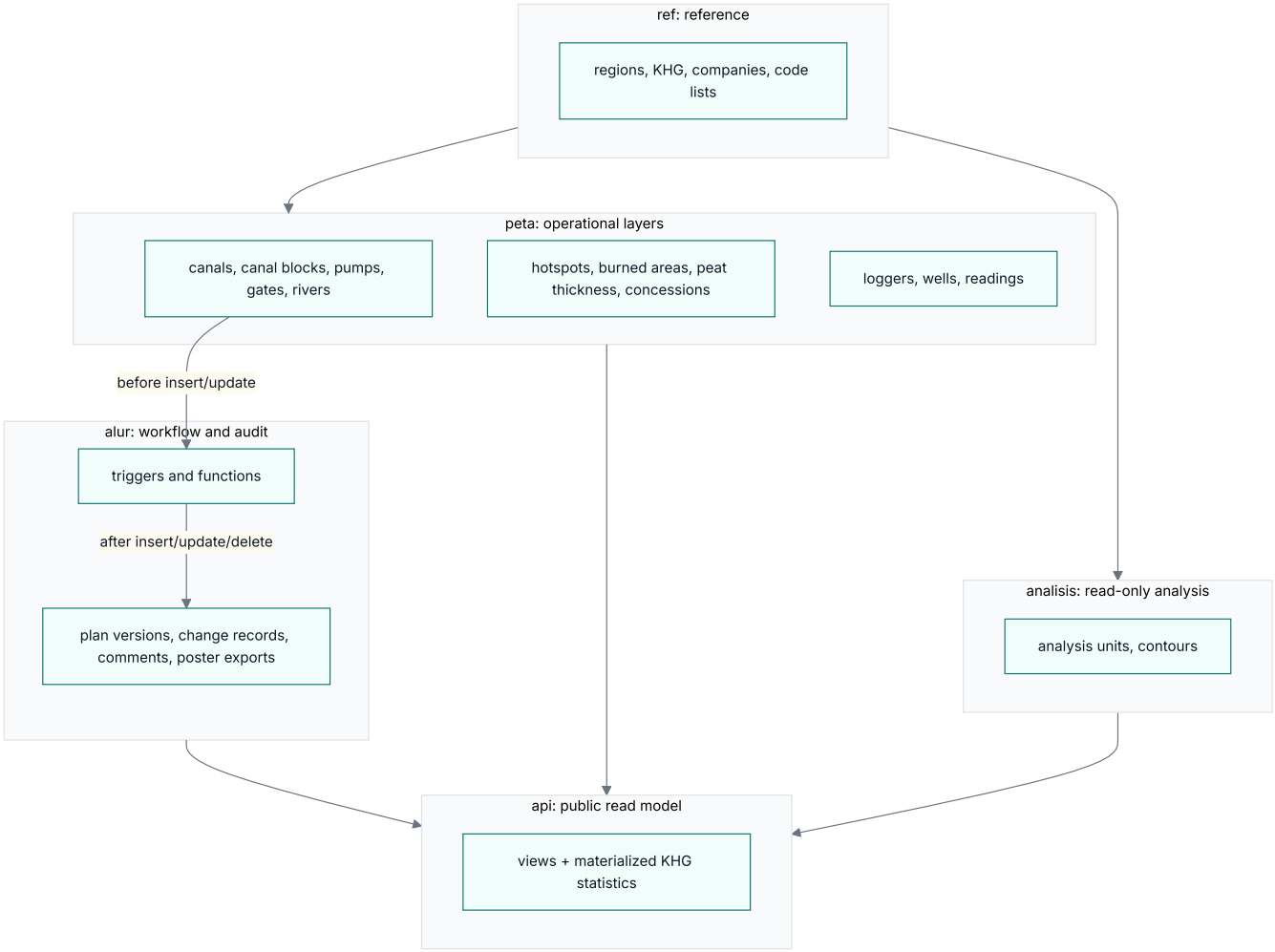


Figure 3. Database schemas and how triggers connect them.

Three triggers run on every editable layer, in this order:

1. `a_meta` **(before)**: fills `khg_id` from the geometry, sets created and updated user, time and channel, and increments `rev`.
2. `b_kode` **(before insert)**: assigns the next code (`P-n`, `SK-n`, `K-n`) inside the KHG.
3. `z_audit` **(after)**: writes a change record with old and new values and links it to the open plan version. It runs as SECURITY DEFINER, so editors cannot write change records themselves.

The writer's identity comes from session settings. The web application sets `app.pengguna` and `app.via` per transaction. QGIS users log in with their own database role, and the API writer role has `app.via = 'api'` as a default.

Data architecture and pipeline

Sources

Source	Format	Features	Notes
Canal blocks BRGM PPEG	5 shapefiles	348,143	28,685 exact duplicates removed; one wrong contour field fixed
BlueBook PEG 2026	File Geodatabase, 7 layers	2,671,331	Analysis units, target villages, canals, contours, hydrological infrastructure
Burned areas 2015 to 2026	Shapefile	584,723	Year columns converted to an array
Hotspots January to September 2026	9 shapefiles in 2 formats	167,718	FIRMS and SiPongi KML exports unified

Pipeline

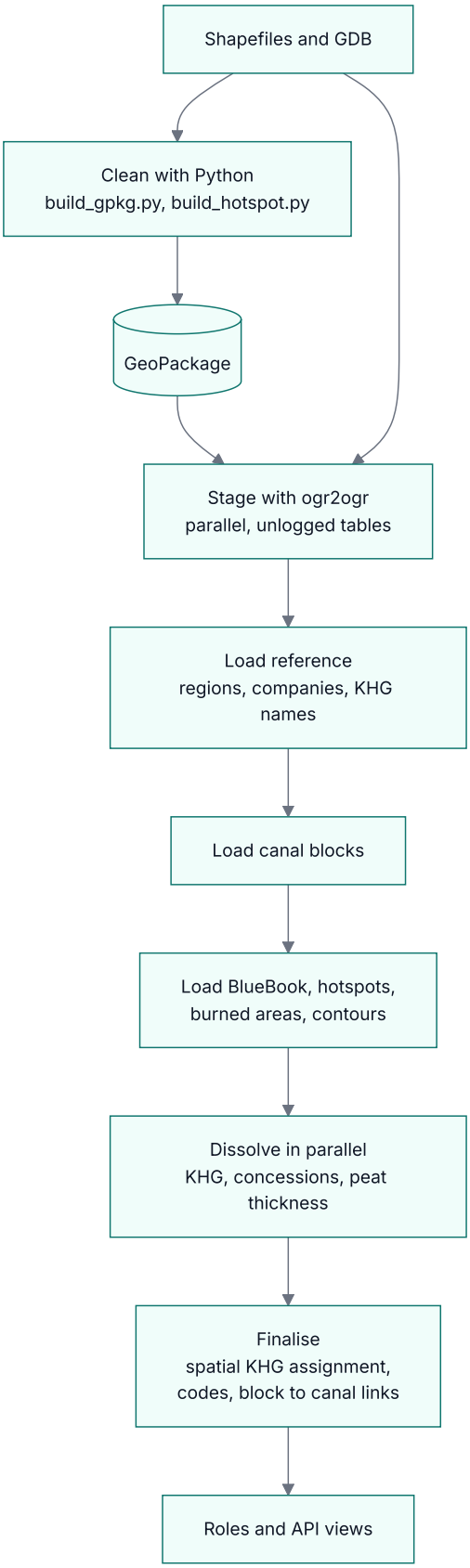


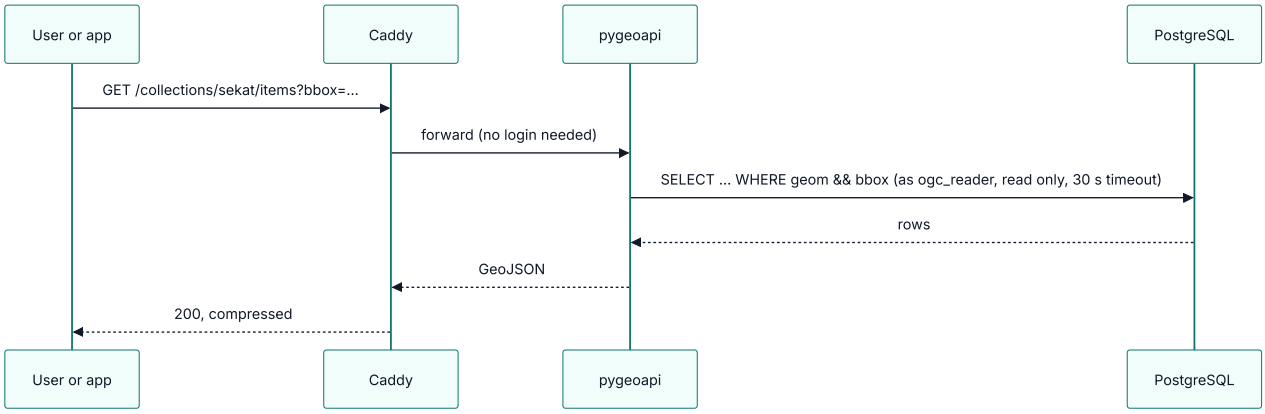
Figure 4. Build pipeline run by db/setup.sh.

Key design points:

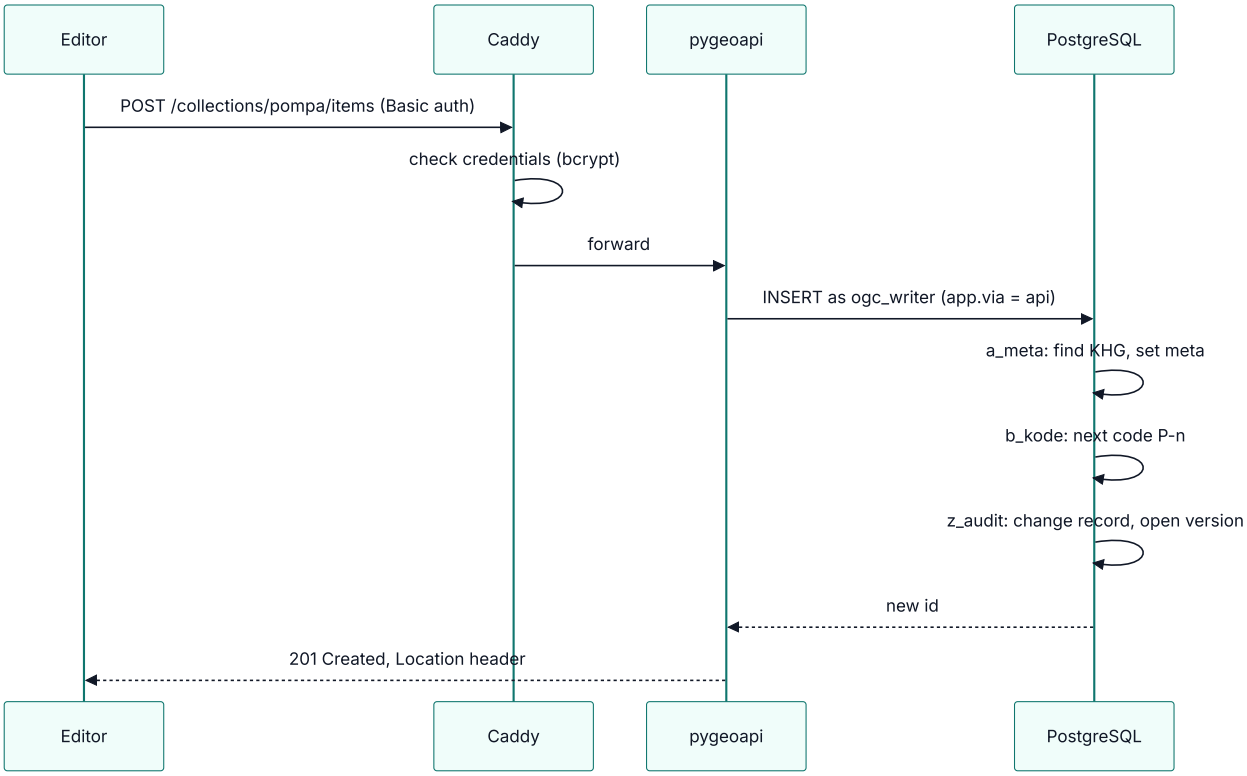
- **Staging first.** Raw data lands in an unlogged `staging` schema, then SQL moves it into the model. Loads run with triggers disabled so that 3.8 million imported rows do not flood the audit log.
- **Coverage union.** KHG boundaries, concessions and peat thickness are dissolved from analysis units with `ST_CoverageUnion`, falling back to `ST_Union` when a result is invalid. All 866 KHG boundaries are valid; their total area differs from the source attribute by 0.3 percent.
- **Subdivided lookups.** KHG polygons are split with `ST_Subdivide` before point-in-polygon joins, which makes KHG assignment for 1.5 million features fast.
- **Spatial linking.** The canal-block source has no canal id (its `Id` column is the contour id), so each block is linked to the nearest canal. 99.85 percent of links match the canal length stated in the source.

Runtime scenarios

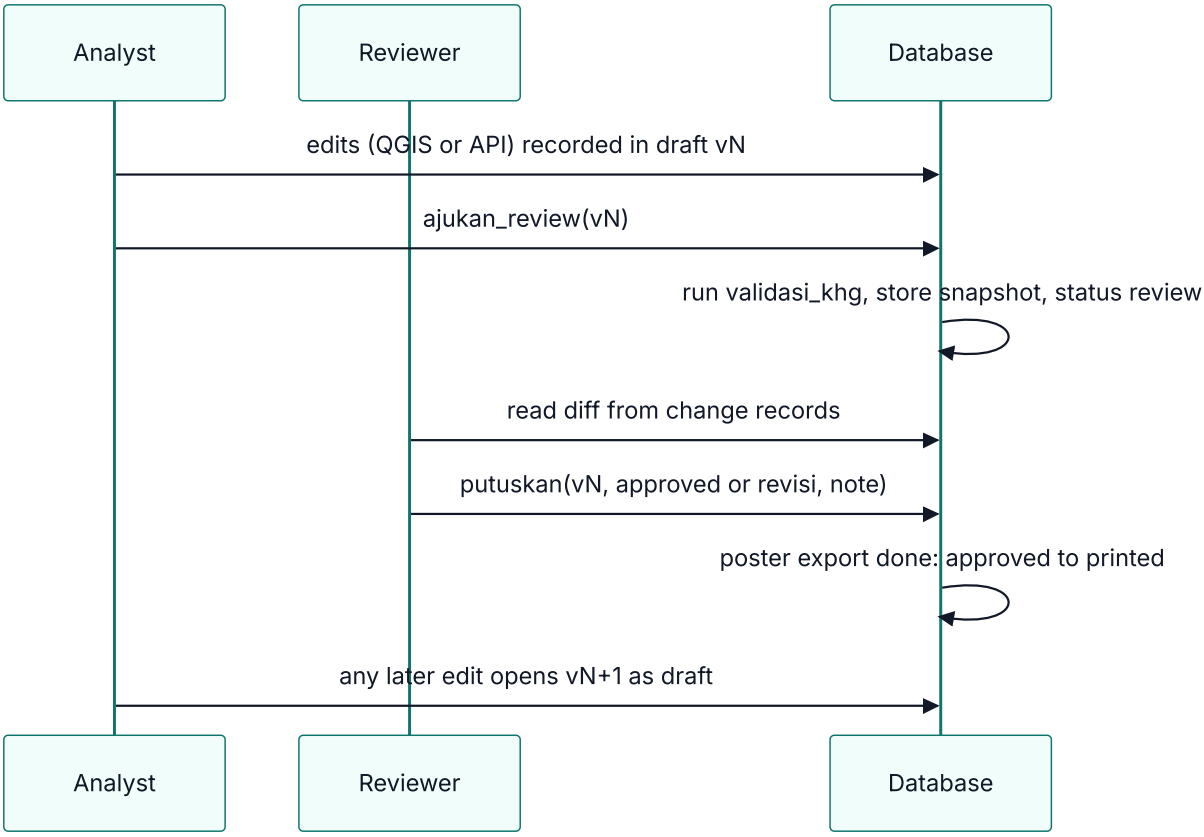
Public read



Authenticated edit through the API



Review workflow



Deployment view

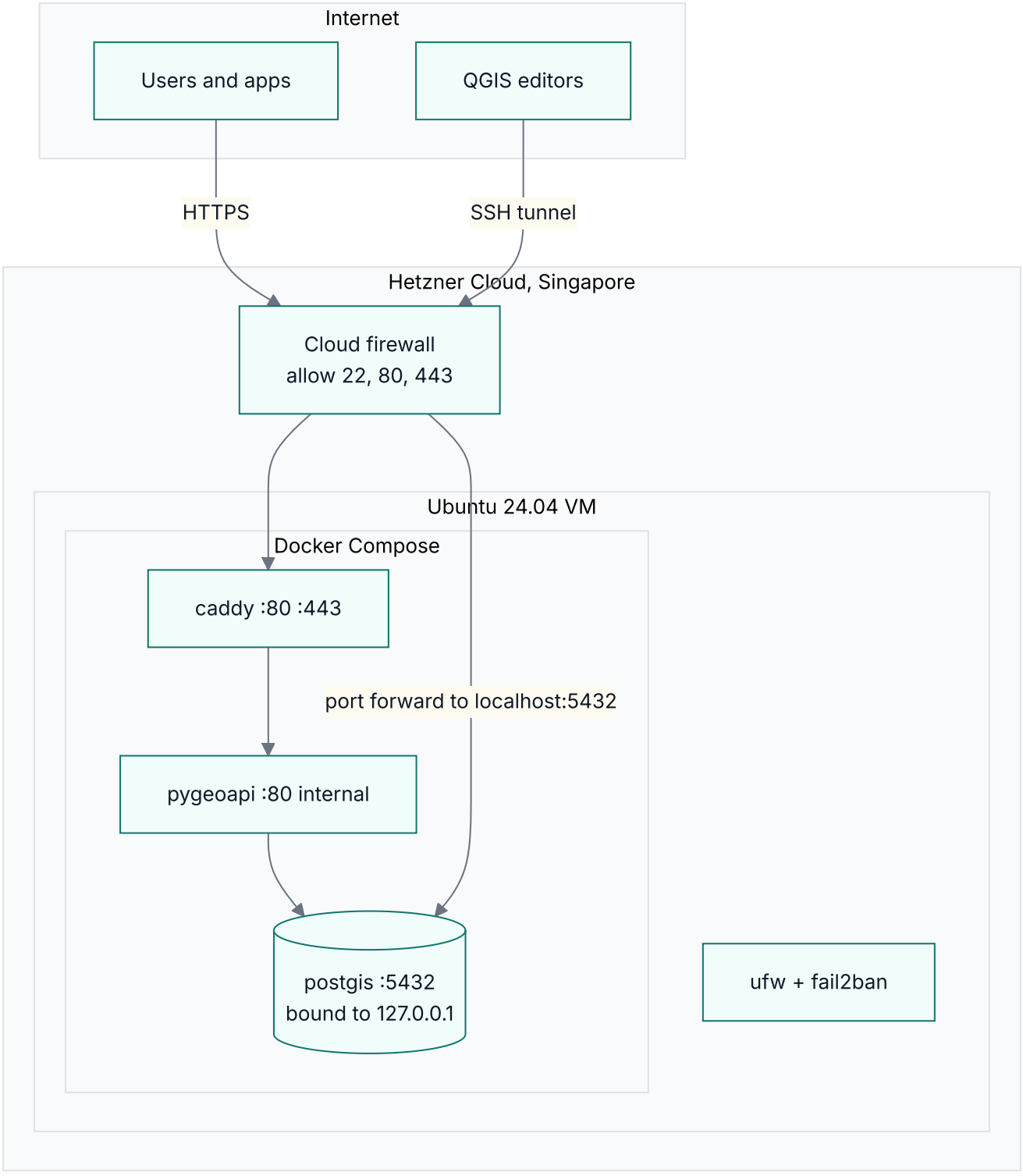


Figure 5. Production style deployment.

Environment	Machine	Notes
Local development	Apple Silicon laptop, Docker Desktop	PostGIS image runs as amd64 through Rosetta; build takes 9.4 minutes

Environment	Machine	Notes
First demo (deleted after use)	Hetzner cpx22, Singapore: 2 vCPU, 4 GB RAM, 80 GB disk	USD 0.0497 per hour, no backups
Shared VM (active)	Existing Hetzner cx23, Nuremberg: 2 vCPU, 4 GB RAM, 40 GB disk, shared with other services	No extra cost; host Caddy serves `https://gambut.178-105-182-162.sslip.io`; containers capped at 1.2 GB (database) and 0.8 GB (API); `deploy/deploy_shared.sh`
Recommended production	Hetzner cpx32, Singapore: 4 vCPU, 8 GB RAM, 160 GB disk	Daily Hetzner backups plus a nightly `pg_dump` kept for 7 days

Deployment is fully scripted:

1. `deploy/provision.sh` creates the SSH key, firewall and server through the Hetzner API. It refuses server types that can no longer be ordered and asks for confirmation before any cost is incurred.
2. `deploy/deploy.sh` waits for cloud-init, syncs files, generates new passwords on the server, uploads and restores the dump (resumable), creates roles, starts the services, obtains TLS certificates and runs smoke tests. It is idempotent and adapts PostgreSQL memory settings to the server size.

Cross-cutting concerns

Security

Layer	Control
Network	Hetzner firewall and ufw allow only ports 22, 80 and 443. The database listens on 127.0.0.1 only.
Transport	HTTPS with automatic Let's Encrypt certificates. Editors reach the database through SSH tunnels.
Authentication	HTTP Basic authentication (bcrypt hash) for writes. SSH keys only; password login and root login are disabled.
Authorisation	Database roles: `gambut_baca` (read), `gambut_edit` (write planning layers), `ogc_reader`, `ogc_writer`, `web_app`, one role per QGIS editor.
Integrity	Audit and versioning functions run as SECURITY DEFINER; editors cannot forge change records.
Secrets	Passwords are random per environment, stored in `.env` files with mode 600 and excluded from Git.
Abuse limits	5,000 features per request, 30 second statement timeout and connection limits for API roles.

Performance (measured)

Operation	Local laptop	Demo server (Singapore)
KHG list with sort and filter	0.15 to 0.35 s	0.49 s server time
Canal blocks in a bounding box	0.06 s	within 1 s end to end
Vector tile (canals, zoom 12)	0.3 s, 24 KB	about 1.3 s including 212 ms network round trip
Full test suite (`db/tests.sql`)	9 s	passed
Full rebuild from source	9.4 min	not needed (restore from dump)
Restore from dump	2 min 48 s	part of a 25 minute deployment

Two optimisations mattered most: spatial prefilters (`&&` with `ST_Expand`) before geodesic distance checks, and planar distance near the equator for very large polygons. Together they cut the validation test from 4 minutes to 9 seconds.

Availability and recovery

Item	Demo	Production target
Topology	Single VM	Single VM
Backups	None	Hetzner daily snapshot + nightly `pg_dump` (7 days) + optional offsite copy
Recovery time	About 25 minutes (reprovision and restore)	About 1 hour
Maximum data loss	Everything since the last dump	24 hours, or minutes with WAL archiving

Observability

Caddy access logs, container logs (`docker compose logs`) and a smoke test after every deployment. An uptime check on the landing page and one collection is recommended for production.

Architecture decisions

No	Decision	Alternatives considered	Reason
AD1	PostGIS as the single source of truth	GeoPackage files, cloud feature services	Concurrent editing, SQL power, QGIS support
AD2	Audit and versioning in database triggers	Application-level logging	QGIS writes directly to the database
AD3	Lookup tables instead of ENUM types	PostgreSQL ENUM	QGIS value relation widgets and easy extension
AD4	pygeoapi for OGC API	GeoServer, pg_featureserv	OGC reference implementation, supports Features Part 4, Tiles and Processes in one service, small footprint
AD5	Separate `api` schema of views	Publish tables directly	Stable public contract with readable names
AD6	Keep national and target analysis units	Keep target units only	Chosen by the product owner; enables national analysis
AD7	Caddy for TLS and auth	nginx with certbot	Automatic certificates and simple configuration
AD8	Hetzner Singapore	GCP Jakarta, DigitalOcean, Contabo	Best cost to reliability ratio with low latency; GCP is 2.5 to 4.5 times more expensive

Risks and technical debt

Risk or debt	Impact	Mitigation
Single server	Downtime during failure	Scripted redeploy in about 25 minutes
Shared API writer account	Change records cannot tell API users apart	Use the web application with per-user identity, or per-editor database roles
pygeoapi PUT requires full geometry and PATCH is not supported	Clients must send complete features	Documented; web client can wrap it
Provisional scoring weights	Priorities may not match policy	Confirm with domain experts
Missing data (rivers, canal status, pumps, loggers, wells, KHG codes)	Some screens show empty values	Structures exist; load when data arrives
Mixed hotspot sources per month	Misleading monthly trends	Re-download all months from one source
Data residency	Government data may need to stay in Indonesia (PP 71/2019)	Same stack can move to GCP Jakarta or a local provider

Cost summary

Option	Per day	Use
Existing shared VM (Nuremberg)	no extra cost	Current deployment
Hetzner cpx22 Singapore, no backup	about EUR 1.19	Dedicated demo server
Hetzner cpx32 Singapore with backup	about EUR 2.7	Recommended production
GCP Compute Engine Jakarta, self-managed	about USD 3.2	If data must stay in Indonesia
GCP Cloud SQL plus small VM, Jakarta	about USD 5.5	Managed database

Prices are Hetzner API list prices including tax in September 2026 and published GCP rates; confirm before budgeting.